**Titre du cours : Mise en place d'un écosystème d'IA**
Code officiel : 420-A57-BB
Professeur : Dr Komi SODOKE

MODULE 03

# Mettre en place un environnement de développement et de déploiement pour l'IA

*Spécialisation technique en Intelligence Artificielle*

**Langages IA** | **IDE & Colab** | **Frameworks ML**

# PLAN DU COURS

**Objectif terminal : Mettre en place un environnement de développement et de déploiement dans le contexte de l'IA**

| | |
|---|---|
| I | **Introduction**<br>Du matériel configuré à l'environnement de développement |
| II | **Langages de programmation pour l'IA**<br>Python · C++ · R · Matlab · Julia — Comparaison et écosystèmes |
| III | **Environnements de développement (IDE)**<br>VS Code · PyCharm · Jupyter · Google Colab — Choix et configuration |
| IV | **Frameworks d'apprentissage machine**<br>PyTorch · TensorFlow · Keras · Scikit-learn · Caffe — Comparaison détaillée |
| V | **Questions de compréhension** |
| VI | **Conclusion & Synthèse** |

I

# Introduction

*Rappel du Module 2 et transition vers l'environnement logiciel*



# I — Rappel : où en sommes-nous ?

Section I

Dans le Module 2, nous avons installé et configuré le socle matériel : système d'exploitation, pilotes GPU, réseau. La machine est prête. Maintenant, il faut y installer les outils de développement.



### Module 2 (fait)

*OS + Drivers + Réseau*

Ubuntu/WSL2, NVIDIA, CUDA, SSH configurés.



### Module 3 (maintenant)

*Langages + IDE + Frameworks*

Python, VS Code, Colab, PyTorch, TensorFlow.



### Module 4 (à venir)

*Virtualisation & Cloud*

Docker, Kubernetes, AWS, GCP.

# I — Objectifs du module

Section I

**À l'issue de ce module, vous serez capable de :**

| | |
|---|---|
| **Maîtriser** | Python et son écosystème scientifique (NumPy, Pandas, Matplotlib, Scikit-learn) pour le développement IA. |
| **Utiliser** | Un IDE adapté à l'IA : VS Code, PyCharm, Jupyter Notebook ou Google Colab selon le contexte. |
| **Installer** | Les frameworks d'apprentissage machine (PyTorch, TensorFlow/Keras) et comprendre leurs différences. |
| **Comparer** | Les langages, IDE et frameworks pour choisir les bons outils selon le projet IA. |
| **Valider** | L'environnement de développement complet via un mini-projet de Machine Learning fonctionnel. |

II

# Langages de programmation pour l'IA

*Python · C++ · R · Matlab · Julia*



# II — Vue d'ensemble des langages IA

Section II

| Langage | Rôle en IA | Forces | Limites | Usage |
|---|---|---|---|---|
| **Python** | Leader incontesté | Syntaxe simple, écosystème immense, support GPU natif | Plus lent que C++ en exécution brute | 90%+ |
| **C++** | Performance brute | Moteur sous-jacent de TensorFlow/PyTorch, temps réel | Complexe, développement plus lent | ~5% |
| **R** | Statistique | Analyse de données, visualisations ggplot2, recherche | Moins adapté au Deep Learning, communauté IA plus petite | ~3% |
| **Matlab** | Ingénierie | Traitement du signal, simulations, toolboxes spécialisées | Payant (licence), écosystème IA limité | ~1% |
| **Julia** | Calcul scientifique | Performances proches de C, syntaxe simple, Flux.jl | Écosystème jeune, communauté plus petite | <1% |



À retenir : Python domine à plus de 90% en IA. C'est le langage que tous les frameworks (PyTorch, TensorFlow, Keras, Scikit-learn) utilisent comme interface principale. Maîtriser Python est obligatoire. Les autres langages ont des rôles complémentaires : C++ pour la performance, R pour la statistique pure.

# II.A — Python : pourquoi est-il n°1 en IA ?

Section II

Python est un langage interprété, dynamiquement typé, créé en 1991 par Guido van Rossum. Sa philosophie : « il devrait y avoir une — et de préférence une seule — manière évidente de faire les choses. »

### Syntaxe lisible

Le code Python ressemble à de l'anglais. Un débutant peut lire et comprendre un script ML en quelques minutes. Moins de code = moins d'erreurs = itérations plus rapides.

### Écosystème scientifique

NumPy (matrices), Pandas (données tabulaires), Matplotlib (graphiques), Scikit-learn (ML classique), PyTorch/TensorFlow (Deep Learning). Tout est en Python.

### Support GPU natif

NVIDIA priorise l'interface Python pour CUDA. PyTorch et TensorFlow utilisent Python pour contrôler les calculs GPU. model.to('cuda') suffit.

### Communauté massive

La majorité des papiers de recherche IA publient leur code en Python. Stack Overflow, GitHub, HuggingFace : tout l'écosystème parle Python.

### L'écosystème Python pour l'IA

| | |
|---|---|
| **NumPy** | Calcul matriciel (base de tout) |
| **Pandas** | Manipulation de données tabulaires |
| **Matplotlib** | Visualisation (graphiques, courbes) |
| **Scikit-learn** | ML classique (classification, régression) |
| **PyTorch** | Deep Learning (Meta/Facebook) |
| **TensorFlow** | Deep Learning + Production (Google) |

# II.A — Python : gestion des bibliothèques (pip & venv)

Section II

**Définitions clés**

pip : le gestionnaire de paquets Python. Permet d'installer, mettre à jour et désinstaller des bibliothèques depuis PyPI (Python Package Index — le « magasin » des bibliothèques Python).

venv : un environnement virtuel qui isole les bibliothèques d'un projet. Chaque projet a son propre venv avec ses propres versions, sans conflit avec les autres projets.

## Commandes essentielles

```
# Créer un environnement virtuel
python3 -m venv mon_projet_ia

# Activer l'environnement
source mon_projet_ia/bin/activate    # Linux/Mac
mon_projet_ia\Scripts\activate       # Windows

# Installer des bibliothèques
pip install numpy pandas matplotlib scikit-learn

# Voir ce qui est installé
pip list
```

Sur Google Colab : pas besoin de venv ni d'installation manuelle. NumPy, Pandas, Matplotlib, Scikit-learn, PyTorch et TensorFlow sont pré-installés. Utilisez !pip install pour ajouter des bibliothèques supplémentaires.

# II.B-C — C++ et R : rôles complémentaires

Section II

## C++

*Le moteur sous le capot*

- TensorFlow et PyTorch sont écrits en C++ sous le capot — Python n'est que l'interface.
- Utilisé quand la vitesse d'exécution est critique : robotique, systèmes embarqués, jeux vidéo.
- CUDA est un langage basé sur C++ — les kernels GPU sont écrits en C++.
- Exemple : un modèle entraîné en Python peut être déployé en C++ via TorchScript pour gagner 10× en vitesse d'inférence.
- Complexe à apprendre mais indispensable pour comprendre ce qui se passe « sous le capot ».

## R

*Le statisticien de l'IA*

- Langage conçu pour la statistique et l'analyse de données. Très utilisé en recherche universitaire.
- ggplot2 : bibliothèque de visualisation considérée comme la meilleure au monde pour les graphiques statistiques.
- tidyverse : écosystème complet pour la manipulation de données (dplyr, tidyr, readr).
- Moins adapté au Deep Learning (pas de support GPU natif aussi mature que Python).
- Idéal pour : tests statistiques, analyses exploratoires, publications scientifiques avec R Markdown.

III

# Environnements de développement (IDE)

*VS Code · PyCharm · Jupyter · Google Colab*



# III — Qu'est-ce qu'un IDE ?

Section III

**Définition**

Un IDE (Integrated Development Environment) est un logiciel qui regroupe tous les outils pour écrire, exécuter, tester et déboguer du code dans une seule interface. C'est votre « atelier complet » de développeur.

### Éditeur intelligent

Coloration syntaxique, autocomplétion, suggestions de code.

### Terminal intégré

Exécuter des commandes sans quitter l'IDE.

### Débogueur

Placer des points d'arrêt, inspecter les variables pas à pas.

### Extensions

Ajouter des fonctionnalités : support Python, Git, Jupyter, Docker.

# III.A-B — VS Code et PyCharm

Section III

## Visual Studio Code

RECOMMANDÉ

*Le plus polyvalent*

- Gratuit et open-source (Microsoft)
- Extensions Python, Jupyter, Docker, Git intégrées
- Remote SSH : coder sur un serveur GPU distant
- IntelliSense : autocomplétion intelligente
- Terminal intégré + débogueur visuel
- Le choix n°1 des développeurs IA dans l'industrie

## PyCharm

PRO

*Le plus professionnel pour Python*

- IDE dédié à Python par JetBrains
- Refactoring avancé et navigation intelligente dans le code
- Tests unitaires intégrés (pytest, unittest)
- Support Docker et bases de données natif
- Version Community gratuite, Pro payante
- Idéal pour les projets Python d'envergure

# III.C — Jupyter Notebook : l'outil des Data Scientists

Section III

> Jupyter Notebook est un environnement interactif qui fonctionne dans le navigateur. Il permet de mélanger du code exécutable, du texte Markdown, des graphiques et des formules LaTeX dans un seul document. C'est LE standard pour l'exploration de données et le prototypage en IA.

| | |
|---|---|
| **Cellules de code** | Exécuter le code par morceaux. Tester une idée sans relancer tout le script. Voir le résultat immédiatement sous la cellule. |
| **Visualisation inline** | Les graphiques Matplotlib, les tableaux Pandas et les images s'affichent directement dans le notebook. |
| **Documentation intégrée** | Cellules Markdown pour documenter chaque étape. Formules LaTeX pour les équations. Idéal pour les rapports de recherche. |
| **Reproductibilité** | Le notebook contient code + résultats + explications. Un collègue peut reproduire exactement votre expérience. |

```
$ pip install jupyter notebook && jupyter notebook    # Lance dans le navigateur
```

*JupyterLab (pip install jupyterlab) : version évoluée avec explorateur de fichiers, terminal, multi-onglets.*

# III.D — Google Colab : Jupyter dans le cloud



### Google Colaboratory (Colab)

Colab est un Jupyter Notebook hébergé gratuitement par Google. Pas d'installation, pas de configuration : tout fonctionne dans le navigateur. Google fournit un GPU Tesla T4 (15 Go VRAM) gratuitement pour chaque session. C'est l'outil idéal pour apprendre le ML sans matériel.

### Avantages

- Gratuit avec GPU Tesla T4
- Zéro installation (navigateur suffit)
- Python, NumPy, Pandas, PyTorch, TF pré-installés
- Sauvegarde automatique sur Google Drive
- Partage facile (comme Google Docs)
- Terminal Linux intégré ( ! commande)

### Limites

- Session limitée (12h max, 90 min d'inactivité)
- GPU pas garanti aux heures de pointe
- Pas d'accès système (pilotes, réseau, SSH)
- Données perdues à la déconnexion (sauf Drive)
- Performances variables selon la charge
- Pas adapté pour la production

IV

# Frameworks d'apprentissage machine

*PyTorch · TensorFlow · Keras · Scikit-learn · Caffe*



# IV — Qu'est-ce qu'un framework ML ?

Section IV

Un framework d'apprentissage machine est une boîte à outils logicielle qui fournit les briques de base pour créer, entraîner et déployer des modèles d'IA : tenseurs, couches de neurones, rétropropagation automatique et optimiseurs. Sans framework, il faudrait tout coder à la main — des milliers de lignes pour un simple réseau de neurones.

*Analogie : un framework est comme une cuisine professionnelle équipée. Au lieu de forger vos couteaux, vous utilisez des outils pro pour vous concentrer sur la recette (votre modèle).*

### Tenseurs & GPU

Structures de données qui tournent sur GPU automatiquement.

### Couches pré-construites

Dense, Conv2D, LSTM, Transformer prêtes à assembler.

### Autograd

Calcul automatique des gradients pour la rétropropagation.

### Optimiseurs

SGD, Adam, AdamW pour ajuster les poids du modèle.

# IV.A — PyTorch (Meta)

Section IV

## PyTorch

*Le framework préféré des chercheurs et de l'industrie*

- Graphe dynamique : le réseau se construit pendant l'exécution. Débogage comme du Python normal (print, breakpoint).
- Autograd : rétropropagation automatique en une ligne — loss.backward(). Chaque opération est tracée.
- Écosystème : torchvision (images), torchaudio (sons), torchtext (NLP), HuggingFace Transformers (LLM).
- GPU natif : model.to('cuda') suffit. Les tenseurs se déplacent entre CPU et GPU facilement.
- Standard de facto : >80% des papiers de recherche utilisent PyTorch. C'est LE framework à maîtriser en 2024+.

```
!pip install torch torchvision    # Sur Colab (déjà pré-installé)
```

**Exemple : vérifier GPU sur Colab**

```
import torch
print(f'PyTorch: {torch.__version__}')
print(f'GPU: {torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU"}')
```

# IV.B — TensorFlow / Keras (Google)

Section IV

## TensorFlow

*Le framework de production*

- Développé par Google Brain, ultra-robuste
- TF Serving : déploiement à grande échelle
- TF Lite : modèles pour mobile et IoT
- TensorBoard : visualisation des entraînements
- Graphe compilé : plus rapide en production

## Keras

*L'interface simplifiée*

- API haut niveau intégrée dans TensorFlow
- Syntaxe ultra-simple : modèle en 5 lignes
- model.fit() : une commande pour entraîner
- Idéal pour débutants et prototypage
- Keras 3.0 : compatible PyTorch, TF et JAX

```
!pip install tensorflow    # Sur Colab (déjà pré-installé)
```

# IV.C — Scikit-learn et autres frameworks

Section IV

## Scikit-learn

Le framework du Machine Learning classique (non Deep Learning). Algorithmes pré-construits : régression, classification, clustering, réduction de dimensionnalité. Interface unifiée : model.fit(X, y) → model.predict(X_new). Idéal pour commencer en ML et pour les problèmes qui ne nécessitent pas de Deep Learning (80% des cas en entreprise !).

```
!pip install scikit-learn    # Pré-installé sur Colab
```

### Autres frameworks notables

| | |
|---|---|
| **Caffe (UC Berkeley)** | Historiquement important pour la vision par ordinateur. Moins actif aujourd'hui, remplacé par PyTorch. Caffe2 a été fusionné dans PyTorch. |
| **JAX (Google)** | Framework de calcul différentiable haute performance. Successeur spirituel de TensorFlow pour la recherche. Utilisé par DeepMind. |
| **HuggingFace Transformers** | Bibliothèque qui fournit des milliers de modèles pré-entraînés (BERT, GPT, LLaMA) prêts à l'emploi. Fonctionne avec PyTorch et TF. |

# IV.D — Comparaison : quel framework choisir ?

Section IV

| Critère | PyTorch | TensorFlow | Keras | Scikit-learn |
|---|---|---|---|---|
| **Créateur** | Meta | Google | Google | INRIA |
| **Facilité** | ★★★★☆ | ★★★☆☆ | ★★★★★ | ★★★★★ |
| **Recherche** | ★★★★★ | ★★★☆☆ | ★★★☆☆ | ★★☆☆☆ |
| **Production** | ★★★★☆ | ★★★★★ | ★★★★☆ | ★★★☆☆ |
| **Deep Learning** | ★★★★★ | ★★★★★ | ★★★★☆ | ✗ |
| **ML classique** | ★★☆☆☆ | ★★☆☆☆ | ★★☆☆☆ | ★★★★★ |
| **GPU** | Excellent | Excellent | Via TensorFlow | CPU only |

## Quelle stratégie adopter ?

**Débutant complet :**
Scikit-learn pour comprendre le ML, puis Keras pour le Deep Learning.

**Étudiant IA / Recherche :**
PyTorch directement. C'est le standard, tous les papiers l'utilisent.

**Production / Déploiement mobile :**
TensorFlow + TF Lite. Mais PyTorch rattrape avec TorchServe et ONNX.

**NLP / LLM :** HuggingFace Transformers (fonctionne avec PyTorch par défaut).

V

# Questions de compréhension

*Testez vos connaissances*



# V — Questions de compréhension

Quiz

1. Pourquoi Python domine-t-il en IA à plus de 90% ? Citez au moins 3 raisons.
2. Quel est le rôle de C++ dans l'écosystème IA, même si on code en Python ?
3. Qu'est-ce qu'un environnement virtuel (venv) et pourquoi est-il important ?
4. Quelle est la différence entre Jupyter Notebook et Google Colab ?
5. Comparez PyTorch et TensorFlow/Keras : quand choisir l'un plutôt que l'autre ?
6. Qu'est-ce que Scikit-learn ? Pourquoi dit-on qu'il couvre 80% des cas en entreprise ?
7. Un débutant veut apprendre le Deep Learning. Quel parcours de frameworks lui recommandez-vous ?
8. Quelles sont les 3 principales limites de Google Colab par rapport à un serveur GPU dédié ?

# IV — Réponses aux questions (1-4)

Quiz

**1** **Pourquoi Python ?**

Python domine en IA parce qu'il est simple à apprendre, possède de nombreuses bibliothèques puissantes et bénéficie d'une grande communauté.

**2** **Rôle de C++**

C++ sert à exécuter les calculs lourds rapidement en arrière-plan, tandis que Python sert d'interface simple.

**3** **Environnement virtuel (venv)**

Un environnement virtuel est un espace isolé qui permet d'installer des bibliothèques sans créer de conflits entre projets.

**4** **Jupyter vs Colab**

Jupyter Notebook fonctionne en local sur la machine, tandis que Google Colab fonctionne en ligne avec accès à un GPU.

# IV — Réponses aux questions (5-8)

Quiz

**5** **PyTorch vs TensorFlow/Keras**

PyTorch offre plus de flexibilité et de contrôle (graphe dynamique, débogage facile). Keras est plus simple d'utilisation (API haut niveau, model.fit()). Pour un débutant, Keras permet de comprendre les concepts rapidement. PyTorch est recommandé ensuite pour la recherche.

**6** **Scikit-learn**

Scikit-learn est une bibliothèque de machine learning classique qui permet de résoudre rapidement la majorité des problèmes en entreprise.

**7** **Parcours débutant Deep Learning**

Un débutant doit apprendre Python, puis Scikit-learn, ensuite Keras, et enfin PyTorch pour aller plus loin.

**8** **Limites de Colab**

Google Colab a des limites de temps d’utilisation, de puissance GPU et de mémoire.

# CONCLUSION



Python est le langage obligatoire en IA. Son écosystème (NumPy, Pandas, Matplotlib) est la base de tout projet.



L'IDE dépend du contexte : VS Code pour le développement, Jupyter/Colab pour l'exploration et le prototypage.



PyTorch est le standard de la recherche. TensorFlow/Keras excelle en production. Scikit-learn couvre le ML classique.



Google Colab permet de commencer immédiatement avec un GPU gratuit — idéal pour l'apprentissage.



L'environnement de développement est maintenant complet. Prochain module : virtualisation (Docker, Cloud).

**Prochain module : Utiliser les technologies de virtualisation dans le contexte de l'IA (Docker, Cloud, Kubernetes)**